



Universidad Nacional de La Plata

Sistemas Paralelos

Explicación práctica

Modelo de programación sobre memoria distribuida
(MPI)

Modelo de programación híbrido
(MPI + Pthreads – MPI + OpenMP)



Facultad de Informática





1

Modelo de memoria distribuida

2

MPI

Funcionamiento. Compilación. Ejecución. Script de ejecución en cluster.

3

Modelo híbrido

Estrategia. Script de ejecución en cluster.

1

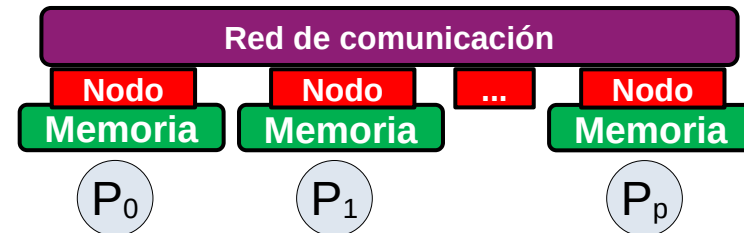
Modelo de memoria distribuida



Modelo de memoria distribuida

4

- **Procesos** independientes, cada uno con su **propio espacio de direcciones**, que se comunican y se sincronizan mediante el intercambio de **mensajes**.
 - No existe memoria compartida entre ellos.
 - Un proceso no puede acceder al espacio de memoria de otro proceso.
 - **No hilos** porque comparten memoria de un mismo proceso y no representan unidades de memoria independiente.
- Surge a partir de los primeros **clusters monocore**.
- Herramientas para el desarrollo de aplicaciones utilizando este modelo:
 - **PVM - 1989**: Impulsó el uso de clusters. **Hoy en desuso**.
 - **MPI**: Estándar vigente. C/C++ - Fortran.
 - Otras: Sockets, RMI, etc.





- **Estándar¹ Message Passing Interface (MPI)**: define sintaxis y semántica de funciones para pasaje de mensajes. **No define los comandos.**
- Varias distribuciones que implementan el estándar en C/C++ y Fortran:
 - OpenMPI
 - Mpich
 - Otros...
 - *Para otros lenguajes (**Java o Python**) son **wrappers** bajo lenguaje **C**.*
- Nace en **1994** como iniciativa de un consorcio, **MPI Forum**:
 - IBM, Intel, Cray Research, Argonne National Laboratory, Universidad de Illinois...
- **SPMD (Single Program, Multiple Data)**: todos los procesos ejecutan el mismo código.
- **Runtime system** o **Runtime environment (ORTE)** gestiona el entorno de ejecución.

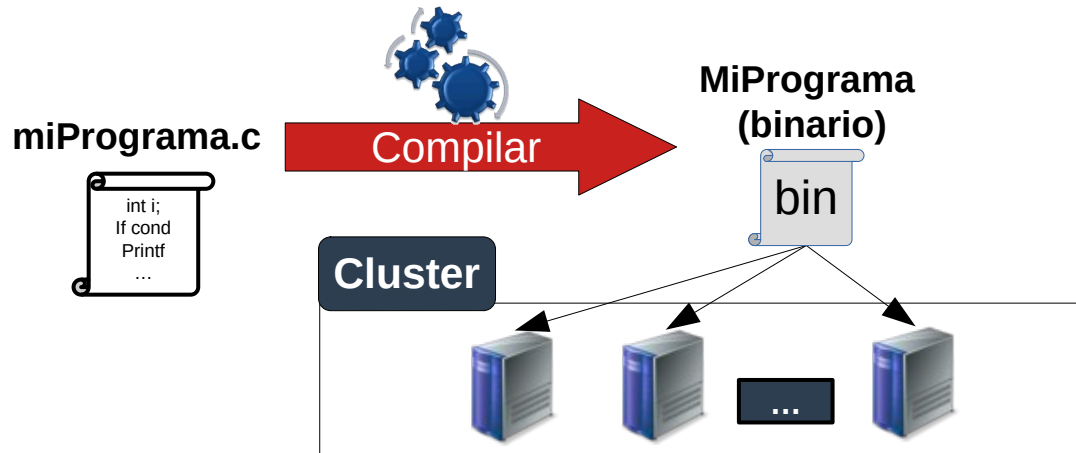
¹<http://www.mpi-forum.org>

MPI

Funcionamiento

7

- Suponemos:
 - Un **cluster** con varios nodos conectados por una red física.
 - Distribución **OpenMPI**.
- Cada nodo debe conocer el **archivo ejecutable (binario)**.
- El programador debe compilar el código fuente y distribuir el binario.
 - MPI no lo hace



MPI

Compilación

8

- Compilación MPI:

```
mpicc -o miPrograma miPrograma.c
```

- La compilación depende de las características del cluster:

Cluster Homogéneo

- Nodos iguales.
- Compilar el código fuente en un nodo.
- Distribuir el binario o compartirlo mediante un sistema de archivos distribuido (**NFS**).

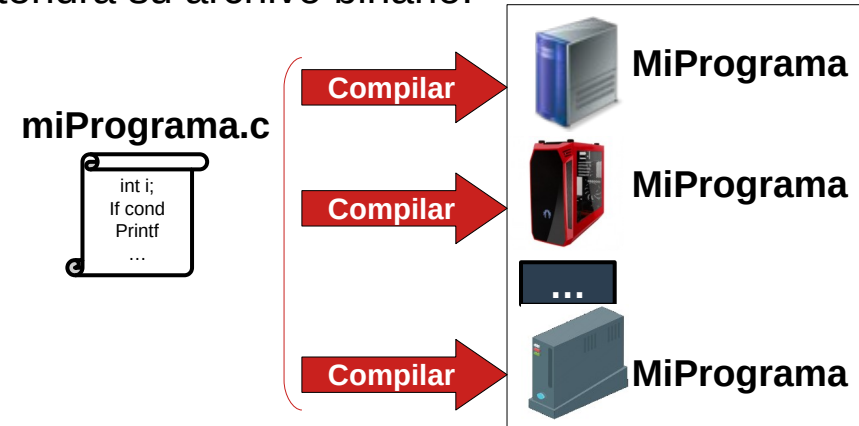
miPrograma.c



Cluster Heterogéneo

- Nodos diferentes.
- Compilar el código fuente en cada nodo. Cada nodo tendrá su archivo binario.

miPrograma.c



MPI

Ejecución

9

- Ejecución MPI:

```
mpirun -np NrProcesos -machinefile maquinas miPrograma
```

- **NrProcesos:** número total de procesos a crear.
- **maquinas:** archivo que contiene el nombre de los nodos.
 - Varía dependiendo de la distribución.

Cluster



Archivo “maquinas”

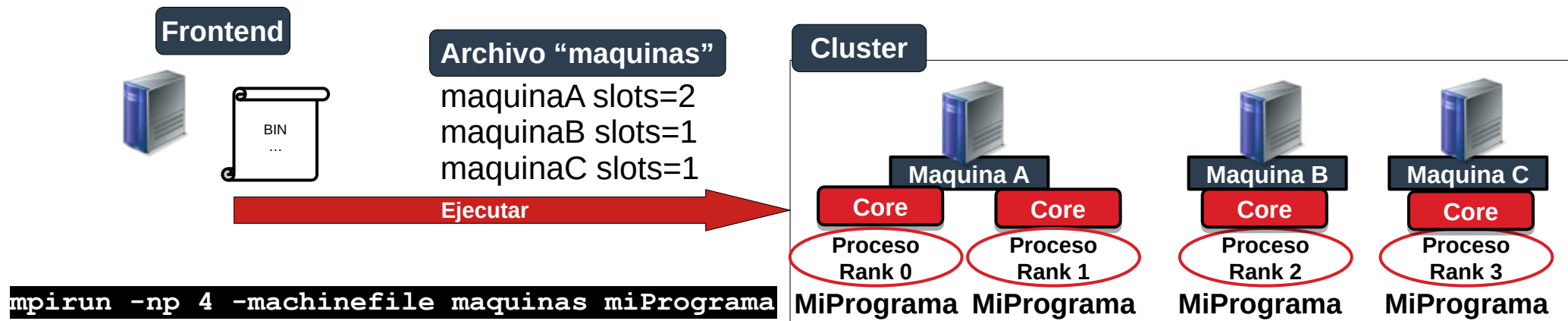
maquinaA slots=2
maquinaB slots=1
maquinaC slots=1

slots: número de procesos que se crearán en cada nodo

MPI Ejecución

10

- Habitualmente, se ejecuta desde un **frontend** (nodo de gestión y servicios)
- El **Runtime System** ejecuta una copia del programa por unidad de procesamiento:
 - **Cada unidad de procesamiento ejecuta una copia del mismo programa**
- Cada proceso recibe un identificador único (**rank**).
 - Generalmente, en el orden del archivo de nodos.



```
tipo_t miVariable;
```

```
int main(int argc, char** argv){  
    int miID;  
    int nrProcs;  
  
    MPI_Init(&argc, &argv);  
    MPI_Comm_rank(MPI_COMM_WORLD, &miID);  
    MPI_Comm_size(MPI_COMM_WORLD, &nrProcs);  
  
    ...  
  
    MPI_Finalize();  
    return(0);  
}
```

Una variable definida en ésta área...

NO ES UNA VARIABLE COMPARTIDA

...entre los procesos.

En un modelo de memoria distribuida:

NO HAY VARIABLES COMPARTIDAS

Recordar:

Cada unidad de procesamiento ejecuta una copia del mismo programa.

Existirá una copia de la variable para cada proceso.

VARIABLE LOCAL Y PRIVADA

Pthreads

Características del script

12

```
#!/bin/bash
```

```
#SBATCH -N 2
```

```
#SBATCH --exclusive
```

```
#SBATCH --tasks-per-node=4
```

```
#SBATCH -o directorioSalida/output.txt
```

```
#SBATCH -e directorioSalida/errors.txt
```

```
mpirun miAplicacionMPI parámetros de programa
```

Número de nodos a reservar.

Procesos por nodo.

Aplicación MPI a ejecutar y sus respectivos parámetros.

```
chmod +x miScript.sh
```

Dar permisos de ejecución.

```
sbatch script
```

Ejecutar con el gestor de recursos.
Retorna un JOBID

NO ejecutar así: `./script`



3

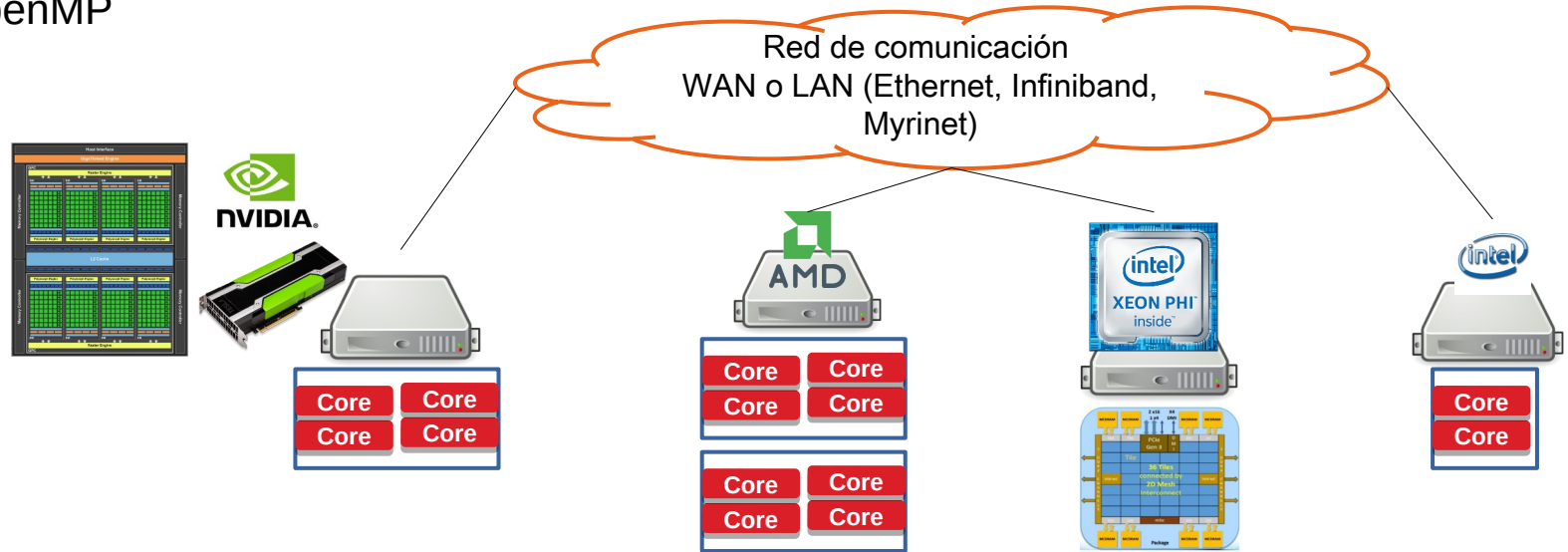
Modelo híbrido

Estrategia. Script de ejecución en cluster.

Modelo de programación híbrido

14

- Actualmente, contamos con una gran diversidad de arquitecturas de cómputo.
- Arquitecturas de memoria compartida y memoria distribuida combinadas formando una **arquitectura híbrida** con mayor potencia de cómputo.
- Podemos combinar:
 - MPI + Pthreads
 - MPI + OpenMP



Modelo de programación híbrido

Estrategia

15

- Estrategia habitual MPI+Pthreads y MPI+OpenMP es:
 - Crear un proceso MPI por nodo del cluster.
 - Cada proceso crea varios hilos en el nodo.

Compilar MPI + Pthreads: `mpicc -lpthread -o mpi_pthreads mpi_pthreads.c`

Compilar MPI + OpenMP: `mpicc -fopenmp -o mpi_omp mpi_omp.c`

Ejecutar como cualquier programa MPI (mpirun) parametrizando el número de hilos.

MPI + Pthreads

Características del script

16

```
#!/bin/bash
```

```
#SBATCH -N 2
```

```
#SBATCH --exclusive
```

```
#SBATCH --tasks-per-node=1
```

```
#SBATCH -o directorioSalida/output.txt
```

```
#SBATCH -e directorioSalida/errors.txt
```

```
mpirun --bind-to none miAplicacionMPI parámetros de programa nroHilos
```

Número de nodos a reservar.

Un proceso por nodo.

Aplicación Híbrida a ejecutar, sus respectivos parámetros y número de hilos.

--bind-to-none para evitar bug de MPI

```
chmod +x miScript.sh
```

Dar permisos de ejecución.

```
sbatch script
```

Ejecutar con el gestor de recursos.
Retorna un JOBID

NO ejecutar así: `./script`

MPI + OpenMP

Características del script

17

```
#!/bin/bash
```

```
#SBATCH -N 2
```

```
#SBATCH --exclusive
```

```
#SBATCH --tasks-per-node=1
```

```
#SBATCH -o directorioSalida/output.txt
```

```
#SBATCH -e directorioSalida/errors.txt
```

```
export OMP_NUM_THREADS=4
```

```
mpirun --bind-to none miAplicacionMPI parámetros de programa
```

Número de nodos a reservar.

Un proceso por nodo.

Hilos por proceso.

Aplicación Híbrida a ejecutar y sus respectivos parámetros.

--bind-to-none para evitar bug de MPI

```
chmod +x miScript.sh
```

Dar permisos de ejecución.

```
sbatch script
```

Ejecutar con el gestor de recursos.
Retorna un JOBID

NO ejecutar así: `./script`

